

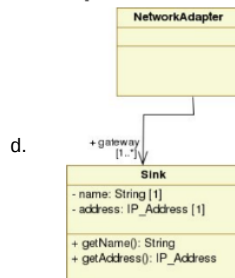
Lezione 6 30/10/23 associazioni + composizioni + aggregazioni + dipendenze

giovedì 2 novembre 2023 10:37

Andiamo ora a vedere i vari tipi di relazioni nel class diagram. **Per relazione si intende la possibilità che una classe A conosca una classe B ed in qualche modo può interagirvi**. Ricordiamo inoltre il problema della generalizzazione. Andiamo a vederle in dettaglio :

1. Associazione

- 
- Rappresenta una relazione tra classi differenti : in dettaglio rappresenta la possibilità a run-time che una classe possa inviare messaggi a quelle associate. Le istanze delle classi possono "comunicare".
- Può essere simmetrica e/o riflessiva



- La relazione gateway può avere un ruolo e può avere una cardinalità. Il nome invece serve per la semantica, ovvero specifica come avviene la relazione.

```
package associazione;
public class Sink {
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

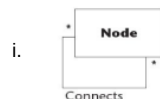
    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

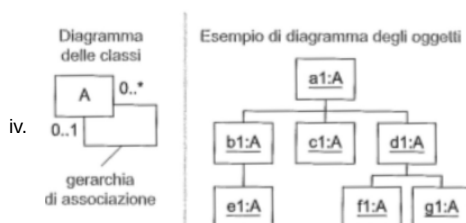
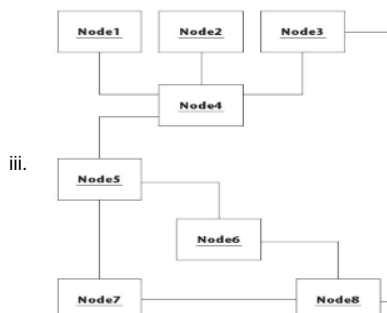
    2 usages
    private String name;
    // type is address
    2 usages
    private String address;
}

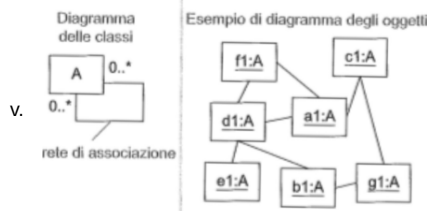
package associazione;
import java.util.Vector;
public class NetworkAdapter {
    private Vector<Sink> gateway;
}
```

f. Vediamo vari esempi :



- Questa relazione di associazione (riflessiva) ha un nome ed un ruolo : attenzione al verso della freccia.

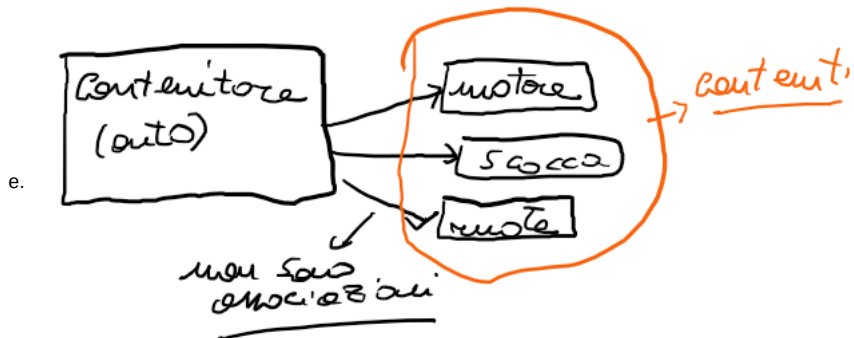




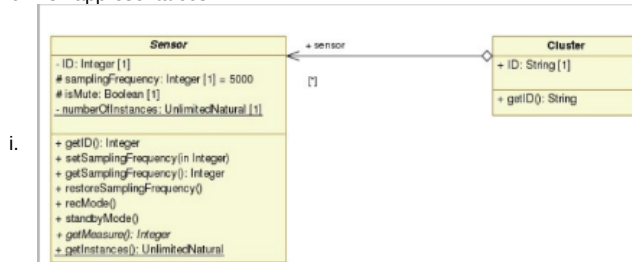
1. Da notare la cardinalità rispetto all'esempio precedente (0...1 contro 0..*).

2. Aggregazione :

- a.
- b. **Denominata anche come whole-part**
- c. Enfatizza la comunicazione tra elemento "contenitore" e classi "contenuto" : comunicazione in modo simmetrico oppure no . Esempio automobile= motore + 4ruote + scocca
- d. In questo tipo di relazione non esiste il ciclo di vita delle istanze coinvolte: nell'esempio cluster - sensor non si ha il ciclo di vita degli aggregati (il sensor può vivere senza cluster): come per esempio nel caso di cancellazione del sensor , oppure un cluster può appartenere a più sensor



- f. In uml si rappresenta così :



- g. In java così :

```
package sensor;

public class Sensor {
    2 usages
    private int id;
    3 usages
    protected int samplingFrequency=5000;
    2 usages
    protected boolean isMute;
    2 usages
    private static int numberOfInstances;

    public int getID() { return id; }
    public void setID(int id) { this.id = id; }
    public int getSamplingFrequency() { return samplingFrequency; }
    public void setSamplingFrequency(int samplingFrequency) { samplingFrequency = samplingFrequency; }
    public boolean isMute() { return isMute; }
    public void setMute(boolean mute) { isMute = mute; }
    public static int getNumberOfInstances() { return numberOfInstances; }
    public void setNumberOfInstances(int numberOfInstances) { this.numberOfInstances = numberOfInstances; }
    public void restoreSamplingFrequency() { this.samplingFrequency=500; }
    public void reset() {}
    2 usages
    public void getMeasure() {}
    2 usages
    public void standbyMode() {}
}

package aggregazione;

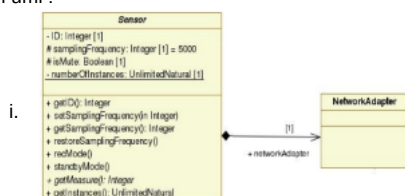
import override_e_overload.C;
import sensor.Sensor;

import java.util.Vector;

public class Cluster {
    2 usages
    private Vector<Sensor> vSensor;
    public Cluster(Vector<Sensor> vSensor) {
        this.vSensor=vSensor;
    }
    public Cluster() {
        this.vSensor=null;
    }
}
```

3. Composizione (aggregazione forte) : da notare che il rombo è pieno.

- a.
- b. **I cicli di vita delle istanze delle classi sono diversi** : possono iniziare in modo indipendente e può capitare che non entrano mai in relazione . Se entrano in relazione invece il contenuto ha stesso ciclo di vita del contenitore.
- c. Per quanto riguarda la distruzione/deallocazione delle istanze , se distruggo istanza contenitore distruggo anche istanza contenuta: sta al programmatore farlo!!
- d. In uml :



e. In java :

```
package composizione;

import associazione.NetworkAdapter;

public abstract class Sensor {
    1 usage
    private NetworkAdapter na;
    public Sensor() {
        this.na=new NetworkAdapter();
    }
}
```

i.

ii. Nell'esempio abbiamo usato una classe astratta di sensor (abstract) , ma in realtà la classe sensor è quella usata in precedenza (polimorfismo e generalizzazione).

iii. Non vi sono riferimenti ad istanza composta.

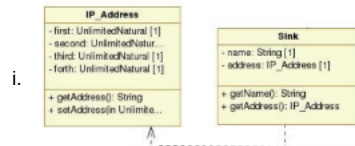
iv. No getter per istanza contenitore.

4. Dipendenza : No mapping diretto in java ed aumenta il livello di accoppiamento tra le classi

a.

b. Rappresenta influenza del cambiamento : se una classe cambia, cambia anche quella associata.

c. In uml :



i.

d. In java :

```
package dipendenza;

public class IPAddress {
    2 usages
    private int first;
    2 usages
    private int second;
    2 usages
    private int third;
    2 usages
    private int fourth;

    2 usages
    private int address;

    public int getFirst() { return first; }
    public void setFirst(int first) { this.first = first; }
    public int getSecond() { return second; }
    public void setSecond(int second) { this.second = second; }
    public int getThird() { return third; }
    public void setThird(int third) { this.third = third; }
    public int getFourth() { return fourth; }
    public void setFourth(int fourth) { this.fourth = fourth; }
    public void setAddress(int address) { this.address=address; }
    public int getAddress() { return address; }
}

package dipendenza;

public class Sink {
    2 usages
    private String name;
    2 usages
    private IPAddress address;

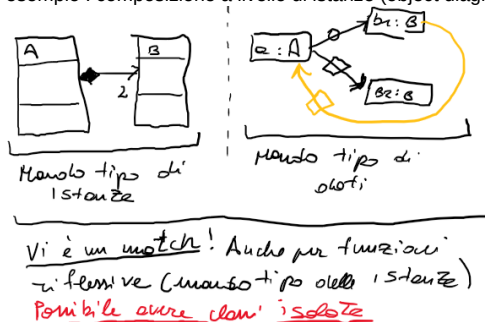
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public IPAddress getAddress() {
        return address;
    }

    public void setAddress(IPAddress address) {
        this.address = address;
    }
}
```

Vediamo un esempio : composizione a livello di istanze (object diagram):



Problema : vogliamo modellare i ruoli organizzativi in un azienda :



a. Che a livello di istanze :

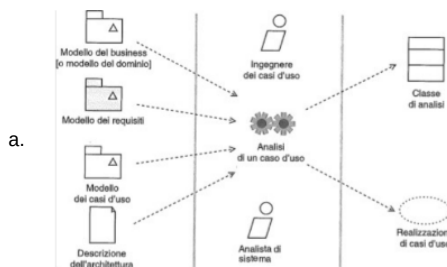
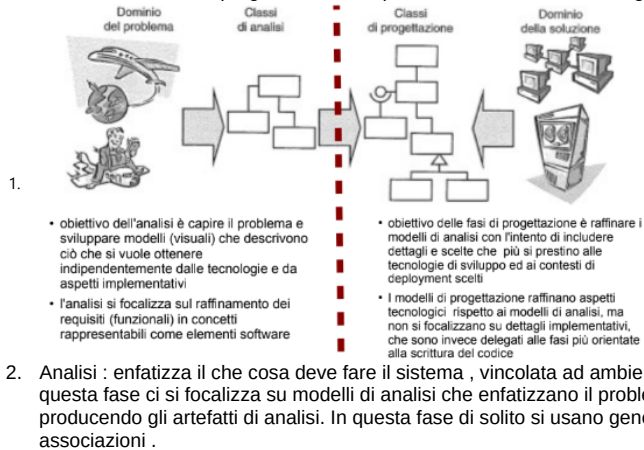
- b. Questa soluzione andrebbe ma, ma come possiamo gestire il fatto di eventuale promozione?? **Uso il pattern della metamorfosi** : cambio ruolo istanza all'interno di una specifica soluzione : disaccoppiando cosa entità rappresenta da cosa entità svolgono.



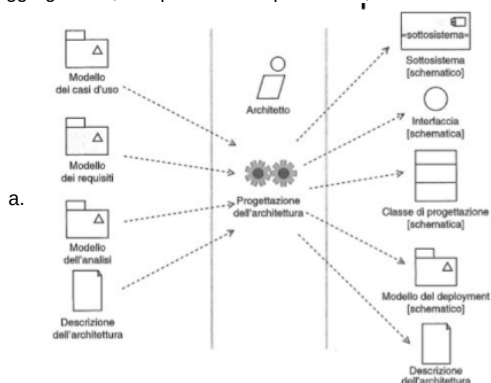
- ii. Andrebbe bene, ma come si fa ad ottenere la promozione??



Torniamo al contesto generale della progettazione e produzione del sw : la produzione viene divisa in due fasi Analisi e progettazione , le quali si avvicinano . In dettaglio :

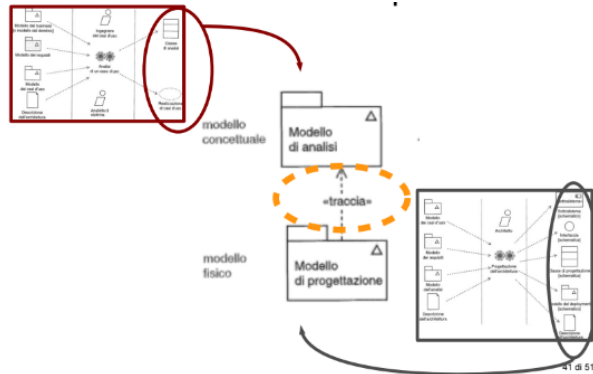


3. Progettazione : raffinano aspetti tecnologici , quindi raffinano gli artefatti della fase di analisi ed inseriscono dettagli "tecnologici" e non "implementativi" . Quindi questa fase si basa sul come verrà risolto il problema producendo gli artefatti di progettazione. In questa fase invece si raffinano le generalizzazioni in realizzazione e le associazioni in aggregazioni , composizioni e dipendenze , a seconda del tipo di relazione

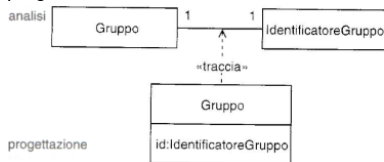


Da notare che tra gli artefatti di analisi e progettazione vi è una relazione : la relazione di traccia . Ricordiamo che entrambi i modelli affrontano le stesse problematiche , ma le analizzano in

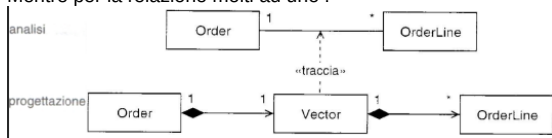
modo differente .



Andiamo a vedere ora come si fanno queste trasformazioni tra fase analisi e quella di progettazione : relazione 1 ad 1



Mentre per la relazione molti ad uno :

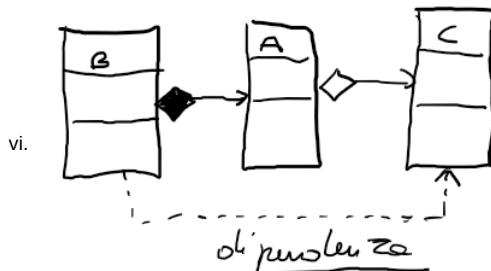


Ricordiamoci l'uml : **E' un mezzo e non una soluzione** : linguaggio per visualizzare, specificare , costruire e documentare i sistemi sw. Non impone nessuna metodologia per il processo di sviluppo (waterfall , Rup , iterative) . **Non basta per vedere se i principi OO vengono correttamente applicati** . Quindi vediamo ora come applicarli correttamente : sviluppiamo il sw , basandoci sulle responsabilità : **RDD (Responsability Driven Design)** : gli oggetti sw sono associati ad una lista di responsabilità sulla struttura (come) e sul comportamento (cosa) . Per quanto riguarda la responsabilità si basa su :

1. Fare :
 - a. Istanziare oggetto
 - b. Dare inizio alle attività
 - i. Coordino attività per arrivare ad un obiettivo comune
 - c. Controllare e coordinare le attività
 - i. Corretta interazione tra oggetti
2. Conoscere :
 - a. Gli oggetti correlati e cose che possono / verranno calcolate (chiede info)
3. Ruoli
 - a. Ruoli che gli oggetti hanno all'interno del sistema
4. Collaborazioni
 - a. Interazioni tra le varie parti del sistema

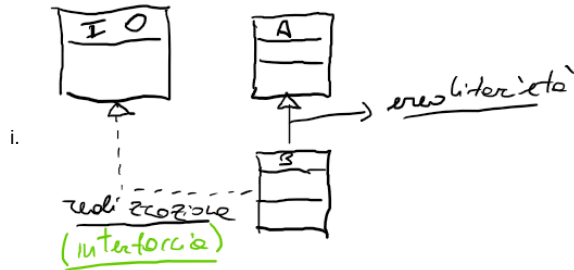
Basandosi su questi principi si arriva alla definizione di GRASP (general responsibility Assignment Software Principles) : **principi di supporto all'apprendimento/progettazione OO** . Si basano sui concetti fondamentali di RDD. Spesso capita che l'applicazione simultanea di tutti questi principi non sia possibile, quindi sta al progettista scegliere opportuna soluzione in base a delle scelte.

1. Creator
 - a. Chi crea ed istanzia oggetto relativo a classe A ?
 - b. Assegno a B questa responsabilità se :
 - i. B aggrega/compone A
 - ii. B registra presso A un contenitore di istanze
 - iii. B utilizza in modo esclusivo A: B interroga/modifica istanze di A
 - iv. B possiede tutti gli elementi per poter inizializzare A
 - v. Le soluzioni alternative devono soddisfare il numero massimo di condizioni.



2. Information expert
 - a. Principio base per assegnare le responsabilità
 - i. Assegno responsabilità se classe B ha tutte le informazioni necessarie su A .
 - ii. Bisogna capire quindi le informazioni/ruolo che classe B riesce a recuperare/calcola/mantiene .
 - b. Questo principio sostiene (condizione non sufficiente per il coupling)
3. Low coupling
 - a. Come ridurre impatto dei cambiamenti?
 - b. Quindi questo principio stima/valuta quanto le classi sono collegate(grado di dipendenza) tra loro : ovvero quanto una classe A conosce la classe B.
 - c. Questo principio viene usato per valutare scelte alternative

d. Se generalizzazione questo accoppiamento è molto alto



e. Il duale di questo prendo il nome di high coesion

4. High coesion

- Correlazioni tra le operazioni di una classe con l'insieme delle responsabilità associate ad essa
- Classi definiscono insieme limitato di operazioni, riferenti ad operazioni altamente correlate.
- Tre tipi di coesione:
 - Bassa
 - Una classe responsabile in molte aree diverse
 - Una classe ha unica responsabilità in aree funzionali diverse del sistema
 - Alta
 - Una classe ha responsabilità moderate in unica area funzionale e collabora con altre classi per arrivare allo scopo comune
 - Moderata
 - Una classe legge in un insieme limitato e scorrelato di aree funzionali diverse, ma che sono logicamente collegate al concetto modellato dalla classe A

5. Single responsibility principle

- Una classe ha un insieme minimo di responsabilità
 - Una classe deve avere il minimo numero di interfacce implementate, così aumenta la probabilità che soddisfacendo le interfacce, soddisfi un'unica responsabilità.

6. Controller

- Quale è il primo oggetto dello strato UI che coordina e riceve le operazioni nel sistema?
- Assegno questa responsabilità ad una classe se :
 - Questa classe rappresenta un punto di accesso complessivo al sistema (facade)
 - Uno scenario all'interno del quale si verifica l'operazione d un sistema

7. Legge di Demetra (principio di conoscenza minima)

- Una classe deve conoscere solo e ciò che le è strettamente necessario
- Interazione solo con classi che conosce direttamente
- Vediamo un esempio:

```

public class Car {
    private Driver d;
    private Vector<Passengers> p;
    public Car(Context context){
        this.d = context.driver;
        this.p = context.getPassengers();
    }
}
  
```

- ii. **Viola la legge di Demetra** : in quanto conosce più di quello che gli serve . Context è information expert di Passenger e Driver.

```

public class Car {
    private Driver d;
    private Vector<Passengers> p;
    public Car(Driver driver){
        this(driver, null);
    }
    public Car(Driver driver, Vector<Passengers> listOfPassengers){
        this.d = driver;
        this.p = listOfPassengers;
    }
}
  
```

- iv. Quindi si opta per questa soluzione

